Complete Demo: Spring Boot + Apache Camel + Kafka

A full, working, end-to-end example with every step included — from project setup to verifying the message in Kafka.

What We're Building

```
Client (curl) --> REST API (Camel) --> Kafka Topic "orders-topic" --> Camel Consumer --> Log
```

1. A Spring Boot app exposes POST /api/orders
2. Camel receives the request, converts it to JSON, and publishes it to a Kafka topic
3. A second Camel route consumes from that same topic and logs the message (simulating a downstream service)

Prerequisites

- Java 17+ installed (java -version)
- Maven installed (mvn -version)
- Docker installed (to run Kafka locally) — docker -version

Step 1: Start Kafka Locally with Docker

Create a file named docker-compose.yml in an empty folder:

```
version: "3.8"
services:
  zookeeper:
    image: confluentinc/cp-zookeeper:7.5.0
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181
      ZOOKEEPER_TICK_TIME: 2000
    ports:
      - "2181:2181"

  kafka:
    image: confluentinc/cp-kafka:7.5.0
    depends_on:
      - zookeeper
    ports:
      - "9092:9092"
    environment:
```

```
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
```

Start it:

```
docker compose up -d
```

Verify both containers are running:

```
docker ps
```

You should see zookeeper and kafka containers listed as Up.

Step 2: Generate the Spring Boot Project

Go to start.spring.io (or use the equivalent curl command below) and generate a project with:

- Project: Maven
- Language: Java
- Spring Boot: 3.x
- Group: com.example
- Artifact: camel-kafka-demo
- Java: 17
- Dependencies: Spring Web

Or generate it from the terminal:

```
curl https://start.spring.io/starter.zip \
  -d dependencies=web \
  -d type=maven-project \
  -d language=java \
  -d bootVersion=3.2.0 \
  -d baseDir=camel-kafka-demo \
  -d groupId=com.example \
  -d artifactId=camel-kafka-demo \
  -o camel-kafka-demo.zip

unzip camel-kafka-demo.zip -d camel-kafka-demo
cd camel-kafka-demo
```

Step 3: Add Camel Dependencies to pom.xml

Open pom.xml and add the Camel BOM (for consistent versions) plus the components we need: camel-spring-boot-starter, camel-kafka-starter, and camel-jackson-starter (for JSON marshalling).

Add this inside <dependencyManagement><dependencies> (create that block if it doesn't exist, placed right after <properties>):

```
<properties>
    <java.version>17</java.version>
    <camel.version>4.4.0</camel.version>
</properties>

<dependencyManagement>
    <dependencies>
        <dependency>
            <groupId>org.apache.camel.springboot</groupId>
            <artifactId>camel-spring-boot-bom</artifactId>
            <version>${camel.version}</version>
            <type>pom</type>
            <scope>import</scope>
        </dependency>
    </dependencies>
</dependencyManagement>
```

Add this inside the existing <dependencies> block (alongside spring-boot-starter-web):

```
<dependency>
    <groupId>org.apache.camel.springboot</groupId>
    <artifactId>camel-spring-boot-starter</artifactId>
</dependency>
<dependency>
    <groupId>org.apache.camel.springboot</groupId>
    <artifactId>camel-kafka-starter</artifactId>
</dependency>
<dependency>
    <groupId>org.apache.camel.springboot</groupId>
    <artifactId>camel-jackson-starter</artifactId>
</dependency>
```

Why each dependency is needed

- **camel-spring-boot-starter —** lets Camel auto-wire into Spring Boot and detect RouteBuilder beans
- **camel-kafka-starter —** gives you the kafka: endpoint component
- **camel-jackson-starter —** gives you .marshal().json() / .unmarshal().json() support

Step 4: Configure Kafka Connection in application.properties

Open src/main/resources/application.properties and add:

```
server.port=8080
```

```
spring.application.name=camel-kafka-demo

# Kafka broker location
camel.component.kafka.brokers=localhost:9092

# Optional: nicer Camel startup logging
camel.springboot.name=CamelKafkaDemo
```

Step 5: Create the Order Model (POJO)

Create src/main/java/com/example/camelkafkademo/model/Order.java:

```
package com.example.camelkafkademo.model;

public class Order {

    private int id;
    private String customer;
    private double amount;

    public Order() {
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getCustomer() {
        return customer;
    }

    public void setCustomer(String customer) {
        this.customer = customer;
    }

    public double getAmount() {
        return amount;
    }

    public void setAmount(double amount) {
        this.amount = amount;
    }
}
```

This is a plain Java object that Camel/Jackson will use to convert incoming JSON into a Java object, and back into JSON before sending to Kafka.

Step 6: Create the Producer Route (REST → Kafka)

Create src/main/java/com/example/camelkafkademo/route/OrderProducerRoute.java:

```
package com.example.camelkafkademo.route;

import com.example.camelkafkademo.model.Order;
import org.apache.camel.builder.RouteBuilder;
import org.apache.camel.model.rest.RestBindingMode;
import org.springframework.stereotype.Component;

@Component
public class OrderProducerRoute extends RouteBuilder {

    @Override
    public void configure() throws Exception {

        // Step 1: Global REST configuration — runs an embedded HTTP server on port 8080
        restConfiguration()
                .component("platform-http")
                .bindingMode(RestBindingMode.json)   // auto convert JSON <-> Java object
                .port(8080);

        // Step 2: Define the REST endpoint
        rest("/api/orders")
                .post("/")
                .type(Order.class)                   // incoming JSON binds to Order.class
                .to("direct:processOrder");

        // Step 3: Process the order and publish it to Kafka
        from("direct:processOrder")
                .routeId("order-producer-route")
                .log("Received order: ${body}")
                .marshal().json()                    // convert Order object -> JSON text
                .to("kafka:orders-topic?brokers=localhost:9092")
                .log("Order published to Kafka topic 'orders-topic'");
    }
}
```

Line-by-line explanation

- **restConfiguration().component("platform-http") —** tells Camel which HTTP engine to use to expose REST endpoints (camel-platform-http-starter is pulled in automatically by camel-spring-boot-starter in recent versions; if you get a missing component error, see the Troubleshooting section)
- **.bindingMode(RestBindingMode.json) —** automatically converts incoming JSON into the Order Java object, and vice versa for responses
- **rest("/api/orders").post("/") —** declares that POST /api/orders is a valid endpoint
- **.type(Order.class) —** tells Camel to bind the incoming JSON body to an Order object
- **.to("direct:processOrder") —** hands off to the second part of the route (kept separate for clarity/reusability)
- **.marshal().json() —** converts the Order Java object back into JSON text (needed because Kafka expects a String/byte payload)

- **.to("kafka:orders-topic?brokers=localhost:9092") —** publishes the JSON message to the Kafka topic orders-topic

> **Note:** If restConfiguration().component("platform-http") throws a "component not found" error, add the camel-platform-http-starter dependency to pom.xml:

```
<dependency>
    <groupId>org.apache.camel.springboot</groupId>
    <artifactId>camel-platform-http-starter</artifactId>
</dependency>
```

Step 7: Create the Consumer Route (Kafka → Log)

This simulates a downstream service reading from the same topic.

Create src/main/java/com/example/camelkafkademo/route/OrderConsumerRoute.java:

```
package com.example.camelkafkademo.route;

import org.apache.camel.builder.RouteBuilder;
import org.springframework.stereotype.Component;

@Component
public class OrderConsumerRoute extends RouteBuilder {

    @Override
    public void configure() throws Exception {

        from("kafka:orders-topic?brokers=localhost:9092&groupId=order-consumer-group")
                .routeId("order-consumer-route")
                .log("Consumed from Kafka: ${body}")
                .unmarshal().json()
                .process(exchange -> {
                    // Example: simulate downstream processing
                    exchange.getIn().setHeader("processedAt", System.currentTimeMillis());
                })
                .log("Processed order with headers: ${headers}");
    }
}
```

Line-by-line explanation

- **from("kafka:orders-topic?brokers=localhost:9092&groupId=order-consumer-group") —** this route is now the consumer; it listens to the orders-topic topic
- **groupId=order-consumer-group —** Kafka consumer group ID; required so Kafka knows how to track this consumer's read position (offset)
- **.unmarshal().json() —** converts the JSON text back into a Java Map
- **.process(exchange -> {...}) —** example of adding custom logic/headers after consuming (you could save to a database here instead)

- **.log(...) —** prints the result to the console so you can see it working

Step 8: Project Structure Check

Your project should now look like this:

```
camel-kafka-demo/
├── docker-compose.yml
├── pom.xml
└── src/
    └── main/
        ├── java/com/example/camelkafkademo/
        │   ├── CamelKafkaDemoApplication.java   (auto-generated by Spring Initializr)
        │   ├── model/
        │   │   └── Order.java
        │   └── route/
        │       ├── OrderProducerRoute.java
        │       └── OrderConsumerRoute.java
        └── resources/
            └── application.properties
```

Step 9: Build and Run the Application

From the project root:

```
mvn clean install
mvn spring-boot:run
```

Watch the console. You should see Camel start up and log both routes:

```
order-producer-route started
order-consumer-route started
```

Step 10: Test the Full Flow

In a new terminal, send a test order:

```
curl -X POST http://localhost:8080/api/orders \
  -H "Content-Type: application/json" \
  -d '{"id":101,"customer":"Asha","amount":250}'
```

What you should see in the application console:

```
Received order: Order[id=101, customer=Asha, amount=250.0]
Order published to Kafka topic 'orders-topic'
Consumed from Kafka: {"id":101,"customer":"Asha","amount":250.0}
```

```
Processed order with headers: {processedAt=1719999999999, kafka.TOPIC=orders-topic, ...}
```

This confirms the full round trip: REST request → Camel producer route → Kafka topic → Camel consumer route → processed.

Step 11: Verify the Message Directly in Kafka (Optional but Recommended)

To prove the message really landed in Kafka (independent of your consumer route), use Kafka's own console consumer tool:

```
docker exec -it $(docker ps -qf "ancestor=confluentinc/cp-kafka:7.5.0") \
  kafka-console-consumer --topic orders-topic \
  --bootstrap-server localhost:9092 \
  --from-beginning
```

You should see the raw JSON message printed:

```
{"id":101,"customer":"Asha","amount":250.0}
```

Press Ctrl+C to exit.

Step 12: Shut Down

When you're done with the demo:

```
# Stop the Spring Boot app: Ctrl+C in its terminal

# Stop Kafka and Zookeeper containers
docker compose down
```

Troubleshooting Checklist

Problem	Likely Cause	Fix
Connection refused on curl	App not running yet	Wait for "Started CamelKafkaDemoApplication" in logs
No resolvable bootstrap urls	Kafka container not up	Run docker ps — confirm kafka is Up; run docker compose up -d again

Problem	Likely Cause	Fix
Component not found: platform-http	Missing dependency	Add camel-platform-http-starter as shown in Step 6 note
Consumer route never prints anything	Wrong groupId or topic name mismatch	Double-check orders-topic spelling matches in both routes
404 on POST	Wrong path or method	Confirm URL is exactly http://localhost:8080/api/orders and method is POST

Quick Recap of the Flow

Step	Component	Action
1	Docker Compose	Starts Kafka + Zookeeper
2–4	Spring Boot + pom.xml	Sets up project and Camel dependencies
5	Order.java	Defines the data shape (POJO)
6	OrderProducerRoute	REST endpoint → JSON → Kafka
7	OrderConsumerRoute	Kafka → JSON → processed log
9–10	mvn + curl	Run and test end-to-end
11	kafka-console-consumer	Independently verify the message in Kafka